



Review of Lectures 1-21



Lecture # 22

Today's Lecture

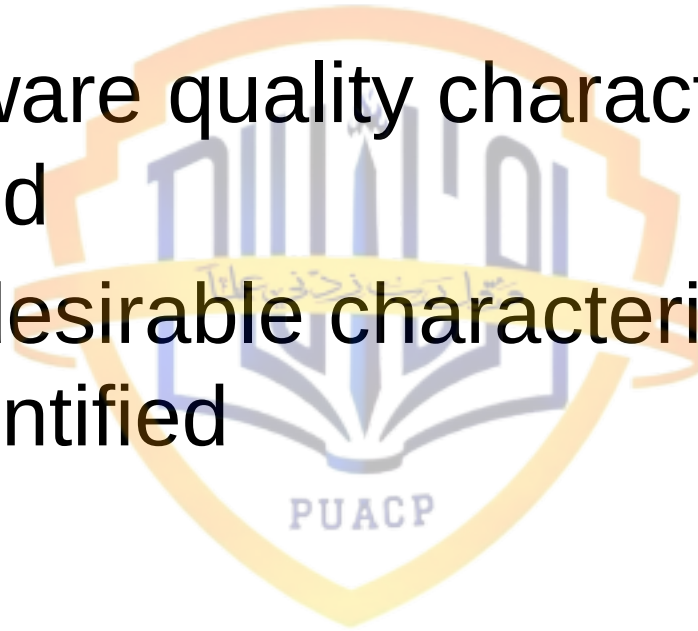
- Today we are going to review the last twenty one lectures. Obviously, it is a very difficult task to cover that much material in one lecture
- So I'll try to mention main points discussed in the last twenty one lectures

Software Quality - 1

- Quality as it relates to all aspects of software (requirements / design / code / tests / documents / training)
- Difficult to define
 - Software quality is somewhat like the concept of beauty. Each of us has a strong opinion about what constitutes beauty, and we recognize it when we see it. But when asked to explain exactly why we regard an object as beautiful, it is hard to put the factors into words

Software Quality - 2

- Good software quality characteristics can be identified
- Bad or undesirable characteristics can also be identified



Software Quality Definitions - 1

- Low levels of defects when deployed, ideally approaching zero
- High reliability, or the capability of running without crashes or strange results
- A majority of clients with high user-satisfaction when surveyed

Software Quality Definitions - 2

- A structure that can minimize “bad fixes” or insertion of new defects during repairs
- Effective customer support when problems do occur
- Rapid repairs for defects, especially for high-severity defects

Why Software Quality? - 1

- Reduces time to market for new products
- Enhances market share compared to direct competitors
- Minimizes “scrap and rework” expenses
- Attracts and keeps “top-gun” personnel
- Minimizes the risk of serious litigation

Why Software Quality? - 2

- Minimizes the risk of serious operating failures and delays
- Minimizes the risk of bankruptcy or business failures, which may be attributed directly to poor quality or poor software quality

What is a Software Defect?

- A software defect is an error, flaw, mistake, failure, or fault in software that prevents it from behaving as intended (e.g., producing an incorrect or unexpected result)
- Software defects are also known as software errors or software bugs

Effects of Software Defects - 1

- Bugs can have a wide variety of effects, with varying levels of inconvenience to the user of the software. Some bugs have only a subtle effect on the program's functionality, and may thus lie undetected for a long time. More serious bugs may cause the software to crash or freeze leading to a denial of service

Effects of Software Defects - 2

- Others qualify as security bugs and might for example enable a malicious user to bypass access controls in order to obtain unauthorized privileges

Effects of Software Defects - 3

- The results of bugs may be extremely serious
- In 1996, the European Space Agency's US \$1 billion prototype **Arian 5** rocket was destroyed less than a minute after launch, due a bug in the on-board guidance computer program

Effects of Software Defects - 4

- In June 1994, a Royal Air Force Chinook crashed into the Mull of Kintyre, killing 29 people. An investigation uncovered sufficient evidence to convince that it may have been caused by a software bug in the aircraft's engine control computer

Categories of Software Defects

- Errors of commission
- Errors of omission
- Errors of clarity and ambiguity
- Errors of speed or capacity



Software Defect Origins

- Errors in Requirements
- Errors in Design
- Errors in Source code
- Errors in User Documentation
- Errors due to “Bad fixes”
- Errors in Data and Tables
- Errors in Test Cases

Software Defect Elimination Strategies

- Effective defect prevention
- High levels of defect removal efficiency
- Accurate defect prediction before the project begins
- Accurate defect tracking during development
- Useful quality measurements
- Ensuring high levels of user-satisfaction

Defect Prevention and Removal

- Both defect prevention and removal techniques are used by the “best-in-the-class” companies
- Defect prevention is very difficult to understand, study, and quantify. We’ll talk about defect prevention in a later lecture
- Both non-test and testing defect removal techniques must be applied

Philosophy of Defect Prevention - 1

- If human misconceptions are the error sources, education and training can help us remove these error sources
- If imprecise designs and implementations that deviate from product specifications or design intentions are the causes for faults, formal methods can help us prevent such deviations

Philosophy of Defect Prevention - 2

- If non-conformance to selected processes or standards is the problem that leads to fault injections, then process conformance or standard enforcement can help use prevent the injection of related faults
- If certain tools or technologies can reduce fault injections under similar environments, they should be adopted

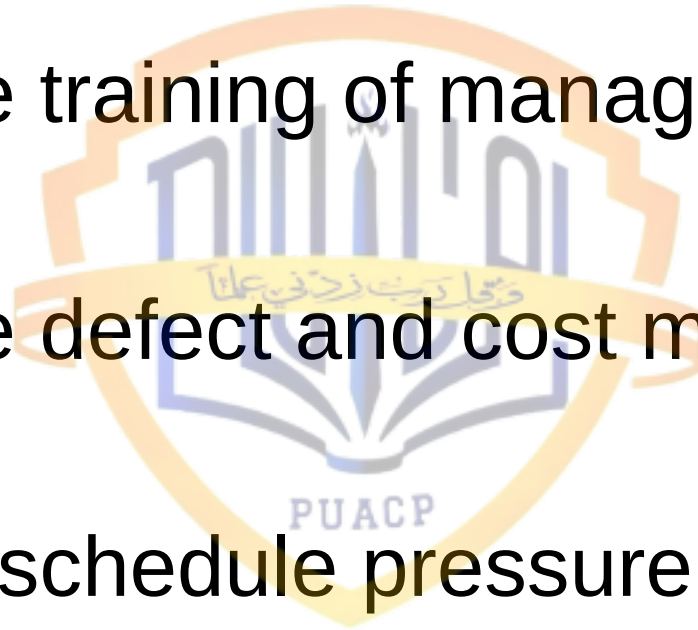
Typical Defect Removal

- Inspections
 - Direct fault detection and removal
- Testing
 - Failure observation and fault removal



Root Causes of Poor Software Quality - 1

- Inadequate training of managers and staff
- Inadequate defect and cost measurement
- Excessive schedule pressure



Root Causes of Poor Software Quality - 2

- Insufficient defect removal
- High complexity levels
- Ambiguous and creeping requirements and design (*feature race & gimmicks*)

Quality Attributes

- Quality attributes set is a way to represent customer quality requirements
- Ask your current and prospective customers about their definition of quality
- Develop a quality assurance program based on the requirements of your customers

Categories of Quality Attributes

- Product-specific quality attributes
- Organization-specific quality attributes



Product-Specific Attributes - 1

- Ease of use
- Documentation
- Defect tolerance
- Defect frequency



Product-Specific Attributes - 2

- Defect impact
- Packaging
- Price versus reliability
- Performance



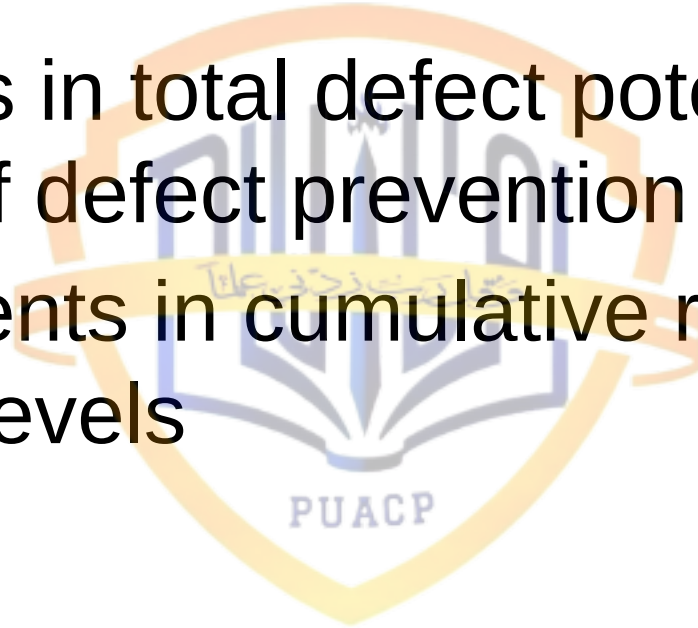
Organization-Specific Attributes

- Service and support
- Internal processes



Two Components of Software Quality Improvement

- Reductions in total defect potentials using methods of defect prevention
- Improvements in cumulative removal efficiency levels



SQA Group - 1

- Every company, which wants to establish a reputation for producing high quality software, must establish a Software Quality Assurance (SQA) Group within the company
- This groups must be funded properly and management must pay attention to the reports and presentations made by this group

SQA Group - 2

- The SQA group report directly to the line-management and not to the project management
- The personnel of the SQA group must work with the project management team, and vice versa to produce high quality software for the company – which is the ultimate goal

SQA Group's Activities - 1

- Preparation of an SQA plan for a project
- Participation in the development of the project's software process description
- Review of software engineering activities to verify compliance with the defined software process

SQA Group's Activities - 2

- Audit of designed software work products to verify compliance with those defined as part of the software process



SQA Group's Activities - 3

- Ensure that deviations in software work and work products are documented and handled according to a documented procedure
- Record any noncompliance and reports to senior management

SQA Plan - 1

- Evaluations to be performed
- Audits and reviews to be performed
- Standards that are applicable to the project
- Procedures for error reporting and tracking

SQA Plan - 2

- Documents to be produced by the SQA group
- Amount of feedback provided to the software project team

Costs of Software Quality





Economics of Software Quality



Economics of Software Quality - 1

- High quality software applications have shorter development schedules than low quality applications because they do not get hung up in integration and testing due to excessive defect levels

Economics of Software Quality - 2

- High quality software applications have lower development and maintenance costs than low quality applications. This is because the cumulative costs of finding and fixing bugs is often the major cost driver for software projects

Economics of Software Quality - 3

- High quality software applications have better reliability levels and longer mean times to failure than low quality applications
- High quality commercial software packages have larger market shares than low quality commercial software packages

Economics of Software Quality - 4

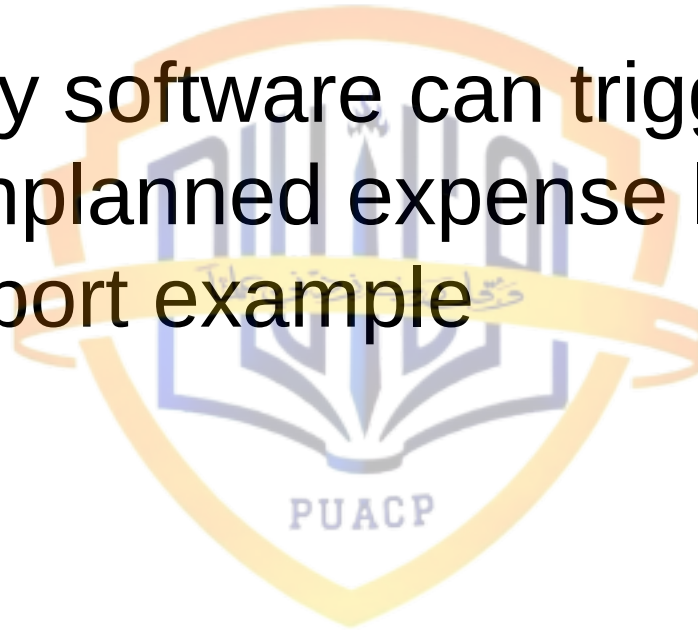
- High quality software achieves better user-satisfaction ratings than low quality software
- High quality software projects score better on employee morale surveys than do low quality software projects

Economics of Software Quality - 5

- High quality software produced under contract or an outsource agreement has a much lower probability of ending up in court for breach of contract or malpractice litigation than low quality software
- High quality software benefits or augments the performance levels of users, while poor quality tends to degrade worker performance

Economics of Software Quality - 6

- Poor quality software can trigger truly massive unplanned expense levels.
Denver airport example



Quality Measurement Categories

- Measurement of defects or bugs in software
 - 100% of software projects
- Measurement of user-satisfaction levels
 - Only for software projects where clients can be queried and actually use the software consciously

Software Defect Quality Measurements - 1

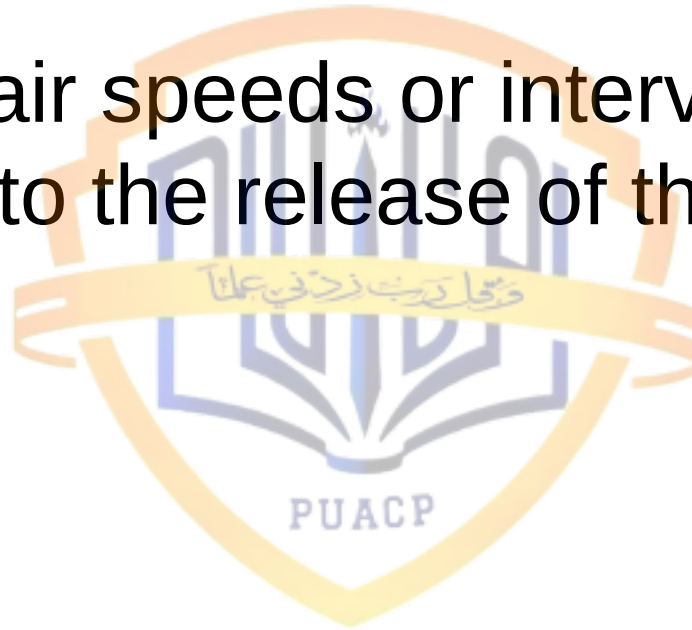
- Defect volumes (by product, by time period, by geographic region)
- Defect severity levels
- Special categories (invalid defects, duplicates, un-duplicatable problems)
- Defect origins (i.e., requirements, design, code, documents, or bad fixes)

Software Defect Quality Measurements - 2

- Defect discovery points (i.e., inspections, tests, customer reports, etc.)
- Defect removal efficiency levels
- Normalized data (i.e., defects per function point or per KLOC)
- Causative factors (i.e., complexity, creeping requirements, etc.)

Software Defect Quality Measurements - 3

- Defect repair speeds or intervals from the first report to the release of the fix



Software User-Satisfaction Quality Measurements - 1

- User perception of quality and reliability
- User perception of features in the software product
- User perception of ease of learning
- User perception of ease of use
- User perception of customer support
- User perception of speed of defect repairs

Software User-Satisfaction Quality Measurements - 2

- User perception of speed of adding new features
- User perception of virtues of competitive products
- User perception of the value versus the cost of the package

Requirements and SQA - 1

- Requirements
 - ☐ Functional
 - ☐ Non-functional
 - ☐ Domain
 - ☐ Inverse
 - ☐ Design constraints
- Requirements engineering process



Requirements and SQA - 2

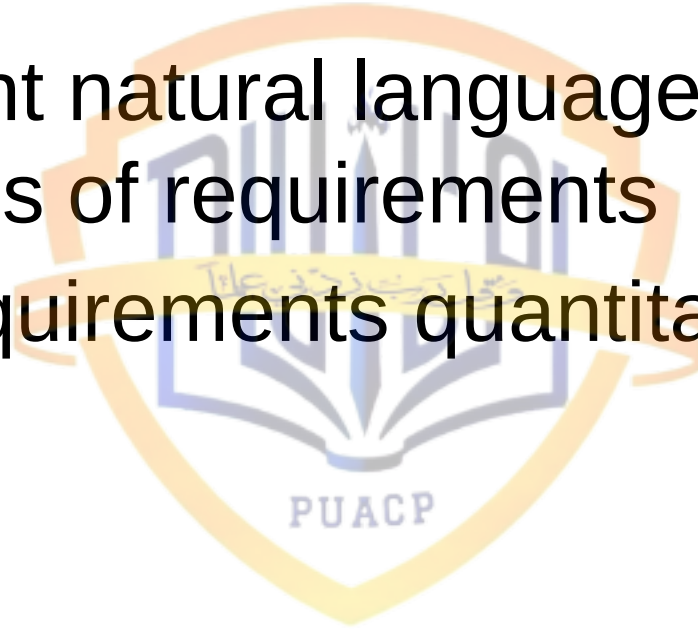
- Requirements defects
- Problems in documenting requirements
- Problems with natural languages
- Guidelines for writing requirements

Guidelines for Writing Requirements - 1

- Define standard templates for describing requirements
- Use language simply, consistently, and concisely
- Use diagrams appropriately

Guidelines for Writing Requirements - 2

- Supplement natural language with other descriptions of requirements
- Specify requirements quantitatively



Additional Guidelines for Writing Requirements - 1

- State only requirement per requirement statement
- State requirements as active sentences
- Always use a noun or a definite pronoun when referring to a thing
- Do not use more than one conjunction when writing requirements statements

Additional Guidelines for Writing Requirements - 2

- Avoid using weak words and phrases. Such words and phrases are generally imprecise and allow the expansion or contraction of requirements beyond their intent

Examples of Words to be Avoided

- About, adequate, and/or, appropriate, as applicable, as appropriate, desirable, efficient, etc., if practical, suitable, timely, typical, when necessary

Additional Guidelines for Writing Requirements - 3

- State the needed requirements without specifying how to fulfill them
- Write complete statements
- Write statements that clearly convey intent

- 
- 
- Requirements document
 - Users of requirements documents
 - System customers
 - Managers
 - System engineers
 - System test engineers
 - System maintenance engineers

Quality Attributes of Requirements Document - 1

- Correct
- Unambiguous
- Complete
- Verifiable
- Consistent



Quality Attributes of Requirements Document - 2

- Understandable by customer
- Modifiable
- Traced
- Traceable
- Design independent



Quality Attributes of Requirements Document - 3

- Annotated
- Concise
- Organized



Software Design and SQA

- Software design and its importance in fostering quality in the work product
- Design defects
- Design model and design process
- Guidelines for design model and design process
- Quality design concepts

Abstraction

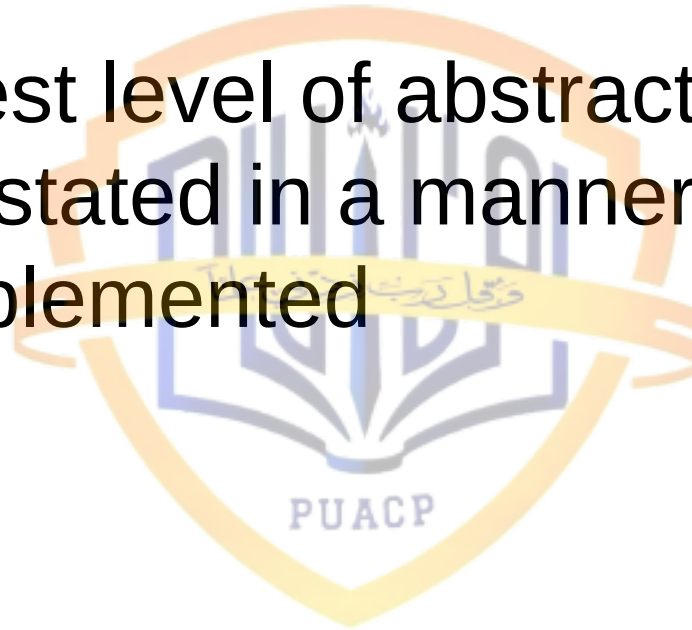
- Abstraction permits one to concentrate on a problem at some level of generalization without regard to irrelevant low-level details
- Abstraction is one of the fundamental ways that we as humans cope with complexity
 - Grady Booch

Levels of Abstraction - 1

- At the highest level of abstraction, a solution is stated in broad terms using the language of the problem environment
- At lower levels of abstraction, a more procedural orientation is taken. Problem-oriented terminology is coupled with implementation-oriented terminology in an effort to state a solution

Levels of Abstraction - 2

- At the lowest level of abstraction, the solution is stated in a manner that can be directly implemented



Refinement - 1

- A program is developed by successively refining levels of procedural detail
- A hierarchy is developed decomposing a macroscopic statement of function in a step-wise fashion until programming language statements are reached
- Refinement is actually a process of elaboration

Refinement - 2

- There is a tendency to move immediately to full detail, skipping the refinement steps. This leads to errors and omissions and makes the design much more difficult to review. Perform stepwise refinement
- Abstraction and refinement are complementary concepts

Modularity - 1

- One of the oldest concepts in software design
- Software is divided into separately named and addressable components, often called, modules, that are integrated to satisfy problem requirements

Modularity - 2

- Modularity is the single attribute of software that allows a program to be intellectually manageable
- Don't over modularize. The simplicity of each module will be overshadowed by the complexity of integration

Information Hiding - 1

- Modules should be specified and designed so that information (procedures and data) contained within a module is inaccessible to other modules that have no need for such information
- IH means that effective modularity can be achieved by defining a set of independent modules that communicate with one another only that information necessary to achieve a software function

Information Hiding - 2

- Abstraction helps to define the procedural (or informational) entities that make up the software
- IH defines and enforces access constraints to both procedural detail within a module and any local data structure used by the module

Information Hiding - 3

- The greatest benefits of IH are achieved when modifications are required during testing and later, during software maintenance
- Both data and procedures are hidden from other parts of the software, inadvertent errors introduced modifications are less likely to propagate to other locations within the software

Cohesion - 1

- Cohesion is the qualitative indication of the degree to which a module focuses on just one thing
- In other words, cohesion is a measure of the relative functional strength of a module
- A cohesive module performs one single task or is focused on one thing

Cohesion - 2

- Highly cohesive modules are better, however, mid-range cohesion is acceptable
- Low-end cohesion is very bad



Coupling

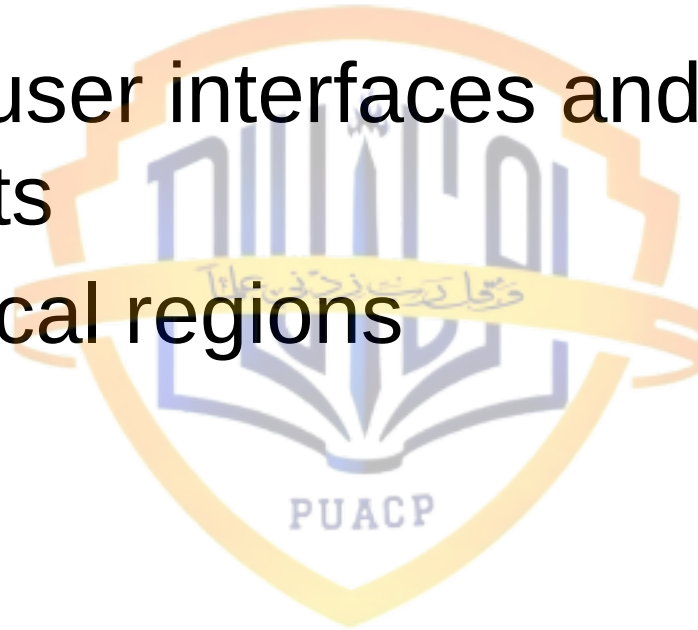
- Coupling is a qualitative indication of the degree to which a module is connected to other modules and to the outside world
- In other words, coupling is a measure of interconnection among modules in a software structure
- Loose coupling is better. Simple connectivity is easier to understand and less prone to “ripple effect”

Quality Practices for General-Purpose Programming - 1

- Use the highest-level programming language possible
- Use integrated development environments
- Adopt a coding standard that prevents common types of defects

Quality Practices for General-Purpose Programming - 2

- Prototype user interfaces and high-risk components
- Define critical regions



What are Reviews?

- A process or meeting during which a work product, or a set of work products, is presented to project personnel, managers, users, or other interested parties for comment or approval. Types include code review, design review, formal qualification review, requirements review, test readiness review

Kinds of Reviews

- Business reviews
- Technical reviews
- Management reviews
- Walk-throughs
- Inspections



Inspections - 1

- An inspection is a rigorous team review of a work product by peers of the producer of the work product
- The size of the team will vary with the characteristics of the work product being inspected; e.g., size, type

Software Inspections - 2

- The primary purpose is to find defects, recording as a basis for analysis on the current project and for historical reference and for improvement for future projects, analyzing them, and initiating rework to correct the defects
- Direct fault detection and removal

Inspections - 3

- Inspections are most effective when performed immediately after the work product is complete, but they can be held any time the work product is deemed ready for inspection

Inspections - 4

- Inspections are critical reading and analysis of software code or other software artifacts, such as designs, product specifications, test plans, etc
- Inspections are typically conducted by multiple human inspectors, through some coordination process. Multiple inspection phases or sessions may be used

Inspections - 5

- Faults are detected directly in inspection by human inspectors, either during their individual inspections or various types of group sessions
- Identified faults need to be removed as a result of the inspection process, and their removal also needs to be verified

Inspections - 6

- The inspection processes vary, but typically include some planning and follow-up activities in addition to the core inspection activity
- Developed by Michael Fagan at IBM and were first reported in public domain in 1976

Inspections - 7

- Inspections remove software defects at reduced cost
- Inspections enable us to remove defects early in the software life cycle, and it always cheaper to remove defects earlier in than later in the software life cycle

Software Inspections Process

- Planning and scheduling
- Overview
- Preparation
- Inspection meeting
- Analysis meeting
- Rework
- Follow-up
- Prevention meeting
- Data recording and reports
- Inspection process monitoring



Inspection Roles

- Moderator
- Producer/Author
- Reader
- Recorder
- Inspector





Practical Issues in Software Inspections

